Cucumber & SoapUI

Complete Interview Preparation Guide

From Fundamentals to Associate Software Engineer Level

Covering L1 & L2 Interview Topics

What's Inside

PART 1: Cucumber & BDD
- Behavior-Driven Development concepts
- Gherkin language & Feature files
- Step Definitions, Hooks, Tags
- Scenario Outlines, Data Tables, Background
- Cucumber with Selenium & Java
- Test Runner & Reports
- 40+ Interview Questions (L1 & L2)

PART 2: SoapUI & API Testing
- Web Services (SOAP vs REST)
- WSDL, XML, JSON fundamentals
- Test Suites, Cases, Steps & Assertions
- Property Transfer & Groovy Scripting
- Data-Driven Testing & Mock Services
- Security & Load Testing
- 35+ Interview Questions (L1 & L2)

Table of Contents

#	Topic	Section
28	Property Transfer	Advanced
29	Groovy Scripting	Advanced
30	Data-Driven Testing	Advanced
31	Mock Services	Advanced
32	Security Testing	Specialized
33	Load Testing	Specialized
34	L1 Interview Questions (Beginner)	Q&A
35	L2 Interview Questions (Intermediate)	Q&A

PART 1

CUCUMBER

Behavior-Driven Development Framework

1. Introduction to BDD

Behavior-Driven Development (BDD) is a software development methodology that evolved from **Test-Driven Development (TDD)**. BDD encourages collaboration between developers, QA testers, and non-technical business participants (Product Owners, Business Analysts) by writing software requirements in a shared, human-readable language.

Why BDD?

- Bridges the gap between business stakeholders and technical teams
- Tests are written in plain English (Gherkin syntax) - understood by everyone
- Focuses on the **behavior** of the application, not just the implementation
- Acts as living documentation - feature files describe what the system does
- Promotes early defect detection and clarity of requirements

TDD vs BDD

Aspect	TDD	BDD
Focus	Implementation (code units)	Behavior (system as a whole)
Language	Programming language	Plain English (Gherkin)
Audience	Developers	Developers, QA, Business
Test Format	Unit tests (assertions)	Scenarios (Given-When-Then)
Goal	Verify code works	Verify business requirements met

2. What is Cucumber?

Cucumber is an open-source BDD testing tool that supports executable specifications written in Gherkin. It reads plain-text feature files and matches each step to executable code (step definitions) written in Java, Ruby, Python, JavaScript, or other languages.

Cucumber Architecture

Cucumber follows a simple workflow:

- **Feature File (.feature)** - Written in Gherkin, contains scenarios in business language
- **Step Definitions** - Java methods that map to Gherkin steps using regex/Cucumber Expressions
- **Test Runner** - Class that tells JUnit/TestNG how to execute feature files
- **Application Code** - The actual code being tested (called from step definitions)
- **Reports** - Generated after execution (HTML, JSON, XML)

Key Features

- Supports multiple languages (Java, Ruby, Kotlin, JavaScript, etc.)
- Integrates with Selenium, Appium, REST Assured, and other tools
- Generates clear, readable reports

- Supports parallel execution
- Encourages reusable code through step definitions

3. Gherkin Language

Gherkin is the language used to write Cucumber feature files. It uses a set of keywords to structure scenarios in a readable format.

Gherkin Keywords

Keyword	Purpose
Feature	Describes a high-level feature of the application
Scenario	A specific example of behavior to test
Given	Sets up the initial context / preconditions
When	Describes an event or action performed by the user
Then	Describes the expected outcome / result
And	Used to add additional Given/When/Then steps
But	Used for negative steps (alternative to And)
Background	Steps that run before every scenario in a feature
Scenario Outline	Template for running the same scenario with different data
Examples	Provides the data table for Scenario Outline
@Tag	Categorizes scenarios for selective execution

Sample Feature File

```
Feature: User Login

  As a registered user
  I want to login to the application
  So that I can access my account

  Scenario: Successful login with valid credentials
    Given the user is on the login page
    When the user enters valid username "john@test.com"
    And the user enters valid password "Pass@123"
    And the user clicks the Login button
    Then the user should be redirected to the dashboard
    And a welcome message should be displayed
```

4. Project Setup (Maven)

To use Cucumber in a Java/Maven project, add these dependencies to **pom.xml**:

```
<dependencies>
  &lt;!-- Cucumber Core --&gt;
  <dependency>
```

```
    <groupId>io.cucumber</groupId>
    <artifactId>cucumber-java</artifactId>
    <version>7.14.0</version>
  </dependency>

  &lt;!-- JUnit Integration --&gt;
  <dependency>
    <groupId>io.cucumber</groupId>
    <artifactId>cucumber-junit</artifactId>
    <version>7.14.0</version>
    <scope>test</scope>
  </dependency>

  &lt;!-- Selenium --&gt;
  <dependency>
    <groupId>org.seleniumhq.selenium</groupId>
    <artifactId>selenium-java</artifactId>
    <version>4.15.0</version>
  </dependency>

  &lt;!-- JUnit 4 --&gt;
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.13.2</version>
  </dependency>
</dependencies>
```

Recommended Project Structure

```
src/
 ■■■ main/
 ■    ■■■ java/
 ■         ■■■ pages/        (Page Object classes)
 ■■■ test/
      ■■■ java/
      ■    ■■■ stepdefinitions/   (Step definition classes)
      ■    ■■■ runners/           (Test runner class)
      ■    ■■■ hooks/             (Before/After hooks)
      ■■■ resources/
           ■■■ features/          (.feature files)
```

5. Feature Files

A **feature file** is a plain text file with the **.feature** extension. It describes a software feature using Gherkin syntax. Each feature file typically contains one Feature and multiple related Scenarios.

Anatomy of a Feature File

```
Feature: <Feature Name>
  <Optional description spanning multiple lines>

  Background:           # Optional: runs before every scenario
    Given <precondition>

  @smoke @login        # Tags
  Scenario: <Scenario Title>
```

```
    Given <step>
    When <step>
    Then <step>
```

Best Practices for Feature Files

- Use business-friendly language - avoid technical jargon
- Keep scenarios short and focused (5-10 steps ideal)
- Each scenario should test ONE behavior
- Use 'I' or 'the user' consistently - don't mix
- Place common preconditions in **Background**
- Use **Scenario Outline** for data-driven tests
- Tag scenarios meaningfully (@smoke, @regression, @login)

6. Step Definitions

A **step definition** is a Java method that connects a Gherkin step to executable code. When Cucumber encounters a step in a feature file, it looks for a matching step definition method to execute.

Sample Step Definition (Java)

```
package stepdefinitions;

import io.cucumber.java.en.Given;
import io.cucumber.java.en.When;
import io.cucumber.java.en.Then;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.By;

public class LoginSteps {

    WebDriver driver;

    @Given("the user is on the login page")
    public void user_on_login_page() {
        driver = new ChromeDriver();
        driver.get("https://example.com/login");
    }

    @When("the user enters valid username {string}")
    public void user_enters_username(String username) {
        driver.findElement(By.id("username")).sendKeys(username);
    }

    @When("the user enters valid password {string}")
    public void user_enters_password(String password) {
        driver.findElement(By.id("password")).sendKeys(password);
    }

    @When("the user clicks the Login button")
    public void user_clicks_login() {
```

```
        driver.findElement(By.id("loginBtn")).click();
    }

    @Then("the user should be redirected to the dashboard")
    public void verify_dashboard() {
        String url = driver.getCurrentUrl();
        assert url.contains("/dashboard");
    }
}
```

Cucumber Expressions vs Regex

Cucumber supports two ways to capture parameters:

```
// Cucumber Expression (recommended - readable)
@Given("the user enters username {string}")
public void enterUsername(String username) { }

@When("the cart has {int} items")
public void cartHasItems(int count) { }

// Regular Expression (more powerful for complex patterns)
@Given("^the user enters username \"(.*?)\"$")
public void enterUsername(String username) { }
```

Built-in Cucumber Expression Parameters

Parameter	Matches	Example
{string}	Text in double quotes	"hello"
{int}	Integer values	42
{float}	Floating point	3.14
{word}	Single word	username
{} (anonymous)	Any text	anything

7. Hooks

Hooks are blocks of code that run before or after each scenario. They are used for setup and teardown - launching the browser, opening a database connection, taking screenshots on failure, etc.

Types of Hooks

Hook	When it Runs
@Before	Before every scenario
@After	After every scenario
@BeforeStep	Before each step in a scenario
@AfterStep	After each step in a scenario
@Before("@tag")	Only for scenarios with the matching tag

Sample Hooks Class

```
package hooks;

import io.cucumber.java.Before;
import io.cucumber.java.After;
import io.cucumber.java.Scenario;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.OutputType;
import org.openqa.selenium.TakesScreenshot;

public class Hooks {

    public static WebDriver driver;

    @Before
    public void setUp() {
        System.out.println("Starting browser...");
        driver = new ChromeDriver();
        driver.manage().window().maximize();
    }

    @After
    public void tearDown(Scenario scenario) {
        if (scenario.isFailed()) {
            byte[] screenshot = ((TakesScreenshot) driver)
                .getScreenshotAs(OutputType.BYTES);
            scenario.attach(screenshot, "image/png", scenario.getName());
        }
        if (driver != null) driver.quit();
    }

    @Before("@db")
    public void connectDatabase() {
        // Runs only for scenarios tagged with @db
        System.out.println("Connecting to database...");
    }
}
```

8. Tags & Tag Expressions

Tags are labels (starting with @) used to organize and selectively run scenarios. They can be applied at Feature level (applies to all scenarios) or Scenario level.

Using Tags

```
@regression
Feature: Shopping Cart

  @smoke @cart
  Scenario: Add product to cart
    Given ...

  @smoke
  Scenario: View cart
    Given ...
```

```
@wip
Scenario: Apply discount coupon
  Given ...
```

Tag Expressions (Runner)

Expression	Meaning
@smoke	Run scenarios tagged @smoke
@smoke and @cart	Run scenarios tagged BOTH @smoke AND @cart
@smoke or @regression	Run scenarios tagged @smoke OR @regression
not @wip	Run all scenarios except @wip
@smoke and not @wip	Run @smoke but exclude @wip

9. Data Tables

Data Tables let you pass structured data (rows and columns) to a step. Useful when a step needs more than just a few simple parameters.

Feature File

```
Scenario: Register multiple users
  Given the following users are registered:
    | username | email             | role  |
    | john     | john@test.com     | admin |
    | jane     | jane@test.com     | user  |
    | bob      | bob@test.com      | user  |
```

Step Definition (Java)

```
import io.cucumber.datatable.DataTable;
import java.util.List;
import java.util.Map;

@Given("the following users are registered:")
public void registerUsers(DataTable dataTable) {
    // As List of Maps (column headers as keys)
    List&lt;Map&lt;String, String&gt;&gt; users = dataTable.asMaps(String.class, Str
ing.class);

    for (Map&lt;String, String&gt; user : users) {
        String username = user.get("username");
        String email = user.get("email");
        String role = user.get("role");
        System.out.println("Registering: " + username);
    }
}
```

DataTable Conversion Methods

Method	Returns
asList()	List of all cell values (1D)

Method	Returns
asLists()	List of List of String (2D)
asMap()	Single Map (for 2-column tables)
asMaps()	List of Maps (headers as keys)

10. Scenario Outline & Examples

A **Scenario Outline** is a parameterized scenario - the same scenario runs multiple times with different sets of data. The data comes from an **Examples** table.

Sample Scenario Outline

```
Scenario Outline: Login with multiple users
  Given the user is on the login page
  When the user enters username "<username>"
  And the user enters password "<password>"
  And clicks login
  Then the result should be "<result>"

  Examples:
    | username       | password   | result   |
    | john@test.com  | Pass@123   | success  |
    | invalid@x.com  | wrong      | failure  |
    | empty          |            | error    |
```

Cucumber will execute the scenario **three times**, once for each row in the Examples table. The values in <angle brackets> are replaced by values from the corresponding column.

11. Background

Background contains steps that run *before every scenario* in a feature file. Use it to set up common preconditions and avoid repetition.

```
Feature: Shopping Cart

  Background:
    Given the user is logged in
    And the user is on the products page

  Scenario: Add a product to cart
    When the user clicks 'Add to Cart' for "Laptop"
    Then the cart count should be 1

  Scenario: Remove a product from cart
    Given a product is already in the cart
    When the user removes the product
    Then the cart should be empty
```

Note: *Use Background sparingly. Too many steps in Background can hurt readability. Each scenario should still be understandable on its own.*

12. TestRunner & Cucumber Options

The **TestRunner** class is the entry point that tells JUnit (or TestNG) to execute Cucumber feature files. It uses the **@CucumberOptions** annotation to configure execution.

Sample TestRunner

```
package runners;

import io.cucumber.junit.Cucumber;
import io.cucumber.junit.CucumberOptions;
import org.junit.runner.RunWith;

@RunWith(Cucumber.class)
@CucumberOptions(
    features = "src/test/resources/features",
    glue = {"stepdefinitions", "hooks"},
    tags = "@smoke and not @wip",
    plugin = {
        "pretty",
        "html:target/cucumber-reports/report.html",
        "json:target/cucumber-reports/report.json",
        "junit:target/cucumber-reports/junit.xml"
    },
    monochrome = true,
    dryRun = false,
    publish = false
)
public class TestRunner { }
```

Cucumber Options Explained

Option	Purpose
features	Path to .feature files
glue	Package(s) containing step definitions & hooks
tags	Tag expression for selective execution
plugin	Reporting plugins (pretty, html, json, junit)
monochrome	Removes ANSI color codes from console output
dryRun	If true, only checks if step defs exist (no execution)
publish	Publishes report to reports.cucumber.io
strict (deprecated)	Fails build if any step is undefined/pending

13. Page Object Model with Cucumber

The **Page Object Model (POM)** is a design pattern that separates page-specific code (locators, actions) into Page classes. Step definitions then call these page methods, keeping them clean and reusable.

Page Class Example

```
package pages;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.FindBy;
import org.openqa.selenium.support.PageFactory;

public class LoginPage {
    WebDriver driver;

    @FindBy(id = "username")
    WebElement usernameField;

    @FindBy(id = "password")
    WebElement passwordField;

    @FindBy(id = "loginBtn")
    WebElement loginButton;

    public LoginPage(WebDriver driver) {
        this.driver = driver;
        PageFactory.initElements(driver, this);
    }

    public void enterUsername(String username) {
        usernameField.sendKeys(username);
    }

    public void enterPassword(String password) {
        passwordField.sendKeys(password);
    }

    public void clickLogin() {
        loginButton.click();
    }
}
```

Step Definition Using POM

```
public class LoginSteps {

    LoginPage loginPage = new LoginPage(Hooks.driver);

    @When("the user enters username {string}")
    public void enterUsername(String username) {
        loginPage.enterUsername(username);
    }

    @When("the user enters password {string}")
    public void enterPassword(String password) {
        loginPage.enterPassword(password);
    }

    @When("the user clicks login")
    public void clickLogin() {
        loginPage.clickLogin();
    }
}
```

14. Cucumber + Selenium WebDriver

Cucumber doesn't perform UI automation by itself - it's a BDD framework. **Selenium WebDriver** is integrated into step definitions to actually drive the browser.

Complete Workflow

- **Feature file** describes the test in plain English
- **Cucumber** reads the feature and finds matching step definitions
- **Step definitions** call **Selenium WebDriver** methods
- **WebDriver** launches a browser and interacts with the application
- **Assertions** (JUnit/TestNG) verify the expected outcomes
- **Hooks** manage browser lifecycle (start/quit)

15. Cucumber Reports

Cucumber supports multiple report formats out of the box:

Report Type	Description
pretty	Console output with colored steps
html	HTML report (clean, browser-viewable)
json	JSON output - input for other tools (Extent Reports, Allure)
junit	JUnit-style XML - used by CI tools like Jenkins
Extent Reports	Third-party rich HTML report (requires extra plugin)
Allure Reports	Third-party report with rich visuals & filters

Configuring Reports

```
plugin = {
    "pretty",
    "html:target/cucumber-reports/report.html",
    "json:target/cucumber-reports/report.json",
    "junit:target/cucumber-reports/junit.xml"
}
```

16. Best Practices

- Write scenarios from the user's perspective - focus on *what*, not *how*
- Keep scenarios independent - one shouldn't depend on another
- Use Background for common setup, but don't overuse it
- Use Scenario Outline for data variations instead of duplicating scenarios
- Reuse step definitions - avoid duplicating step phrases
- Use Page Object Model for maintainable UI code

- Use meaningful tags (@smoke, @regression, @api, @ui)
- Take screenshots on failure - add to scenario via Scenario.attach()
- Keep step definitions short - delegate logic to helper classes
- Don't put assertions in feature files - put them in step definitions
- Use proper version control - commit feature files separately if needed
- Run smoke tests before regression - use tags to control scope

17. L1 Interview Questions (Cucumber)

Level 1 questions test fundamental understanding of Cucumber concepts.

Q1. What is Cucumber?

Cucumber is an open-source BDD (Behavior-Driven Development) testing tool that lets you write test cases in plain English using the Gherkin language. It bridges the gap between developers, testers, and business stakeholders by making test specifications human-readable.

Q2. What is BDD and how is it different from TDD?

BDD (Behavior-Driven Development) focuses on the **behavior** of the application from the user's perspective, written in plain English. TDD (Test-Driven Development) focuses on **unit-level testing** written in code by developers. BDD encourages collaboration; TDD is developer-centric.

Q3. What is Gherkin?

Gherkin is the language used by Cucumber to define test cases. It uses keywords like **Feature, Scenario, Given, When, Then, And, But** to describe test behavior in a structured, human-readable format.

Q4. What are the main Gherkin keywords?

Feature (describes the feature), **Scenario** (a test case), **Given** (precondition), **When** (action), **Then** (expected result), **And/But** (additional steps), **Background** (common steps), **Scenario Outline** (parameterized scenario), **Examples** (data for outline).

Q5. What is a feature file?

A feature file is a plain-text file with a **.feature** extension that contains one Feature and one or more Scenarios written in Gherkin. It describes a piece of functionality from the user's perspective.

Q6. What is a Step Definition?

A Step Definition is a Java (or other language) method annotated with @Given, @When, or @Then that maps to a Gherkin step. When Cucumber encounters a step in the feature file, it executes the matching step definition.

Q7. What is the use of the Background keyword?

Background is used to define a set of steps that should be executed **before every scenario** in a feature file. It avoids repetition of common preconditions like login or navigation.

Q8. What is a Scenario Outline?

Scenario Outline is a way to run the same scenario multiple times with different data. The data is provided in an **Examples** table below the scenario. Variables are denoted using <angle brackets>.

Q9. What is the use of Tags in Cucumber?

Tags (e.g., @smoke, @regression) are used to categorize scenarios. You can selectively run scenarios by tag using the **tags** option in the TestRunner or via command line.

Q10. What are Hooks in Cucumber?

Hooks are special methods annotated with **@Before** and **@After** that run before and after every scenario. They are used for setup (launching browser) and teardown (closing browser, taking screenshots).

Q11. What is the use of the 'glue' option in TestRunner?

The 'glue' option specifies the **package(s)** where Cucumber should look for step definitions and hooks. Without it, Cucumber won't be able to find matching step methods.

Q12. What is dry run in Cucumber?

When **dryRun = true**, Cucumber checks whether **step definitions exist** for all Gherkin steps without actually executing them. It's useful for verifying step coverage.

Q13. What languages does Cucumber support?

Cucumber supports Java, Ruby, JavaScript, Kotlin, Python (via Behave), .NET (SpecFlow), Scala, and many others.

Q14. What is the file extension for a feature file?

.feature

Q15. Can we have multiple scenarios in one feature file?

Yes, a feature file can have multiple scenarios as long as they are related to the same feature.

Q16. What is a Test Runner class?

A Test Runner is a Java class that uses @RunWith(Cucumber.class) and @CucumberOptions annotations to tell JUnit/TestNG how to find and execute feature files and step definitions.

Q17. What is the difference between @Before and @BeforeStep?

@Before runs once before each **scenario**. **@BeforeStep** runs before **every individual step** within a scenario.

Q18. What is a Data Table in Cucumber?

A Data Table is used to pass **structured data** (rows and columns) to a step. It's represented in Gherkin using pipe characters | and can be converted to List, Map, or List of Maps in step definitions.

Q19. What is the use of plugin in @CucumberOptions?

The plugin option specifies the reporting format(s) for test results - pretty, html, json, junit, etc. Multiple plugins can be combined.

Q20. Can we run a single scenario from a feature file?

Yes, by using tags. Tag a specific scenario (e.g., @runMe) and specify **tags = "@runMe"** in @CucumberOptions, or use Cucumber's IDE plugin to run a single scenario directly.

18. L2 Interview Questions (Cucumber)

Level 2 questions test deeper understanding including frameworks, integration, and real project scenarios.

Q1. How do you implement the Page Object Model with Cucumber?

Create separate Page classes (e.g., LoginPage, HomePage) that contain page-specific locators (WebElements) and methods. Initialize them in step definitions or hooks. The step definitions then call page methods instead of directly using Selenium commands. This separates UI logic from test logic and improves maintainability.

Q2. How do you share data between step definitions?

Multiple approaches: **(1)** Create a singleton or shared context class. **(2)** Use **PicoContainer** (or other DI tools) - Cucumber's official dependency injection that automatically shares object instances between step definition classes within the same scenario. **(3)** Use ThreadLocal variables for parallel execution. **(4)** Use Scenario context as a Map for storing intermediate values.

Q3. How do you handle dependent scenarios in Cucumber?

Scenarios in Cucumber should ideally be **independent**. If sequential dependency is required, options include: **(1)** Use Background to set up common state. **(2)** Use hooks to set up data via API/database before each scenario. **(3)** Combine related steps into one scenario. **(4)** Avoid stateful coupling - prefer building required state through API calls in @Before hooks rather than depending on previous UI scenarios.

Q4. How can you achieve parallel execution in Cucumber?

Cucumber 4+ supports parallel execution natively. Options: **(1)** With **JUnit 4** - use the maven-surefire-plugin with **parallel=methods** and **forkCount**. **(2)** With **JUnit 5** - use cucumber-junit-platform-engine with junit-platform.properties. **(3)** With TestNG - use the Cucumber-TestNG adapter with @DataProvider parallel=true. WebDriver must use ThreadLocal to avoid conflicts across threads.

Q5. What is the difference between Cucumber Expressions and Regular Expressions?

Cucumber Expressions (e.g., {string}, {int}, {word}) are **simpler and more readable** - introduced in Cucumber 3+. Regular Expressions are **more powerful** for complex pattern matching but harder to read. You cannot mix both in a single project for the same step. Cucumber Expressions are the recommended modern approach.

Q6. How do you take screenshots on test failure?

In an @After hook, check if **scenario.isFailed()**. If true, cast the driver to **TakesScreenshot**, capture as bytes, and attach to the scenario using **scenario.attach(bytes, "image/png", scenarioName)**. The screenshot will appear in the HTML report automatically.

Q7. What is the difference between asMap() and asMaps() for DataTable?

asMap(Class, Class) converts a 2-column data table into a single Map (key-value pairs). **asMaps(Class, Class)** converts a table with header row into a **List of Maps**, where each row becomes a Map with column headers as keys. Use asMap for simple key-value data, asMaps for tabular records.

Q8. How do you handle tag expressions in Cucumber?

Tag expressions use logical operators: **and, or, not**. Examples: **@smoke and @ui** (both tags), **@smoke or @regression** (either tag), **not @wip** (exclude WIP), **@smoke and not @flaky** (smoke but not flaky). Configured in @CucumberOptions or via command line

-Dcucumber.filter.tags.

Q9. How do you integrate Cucumber with Maven?

Add cucumber-java, cucumber-junit (or cucumber-testng) dependencies in pom.xml. Use maven-surefire-plugin to execute tests. Run **mvn test** to execute all features. Use **mvn test -Dcucumber.filter.tags="@smoke"** to run specific tags. Use maven-cucumber-reporting plugin for enhanced reports.

Q10. What is the role of cucumber-jvm?

cucumber-jvm is the Java implementation of Cucumber. It contains modules like cucumber-core (engine), cucumber-java (annotations), cucumber-junit (JUnit integration), and cucumber-testng (TestNG integration). It executes feature files written in Gherkin by mapping them to Java methods.

Q11. How do you generate Extent Reports with Cucumber?

Add the **extentreports-cucumber-adapter** dependency. Configure the plugin in @CucumberOptions: **"com.aventstack.extentreports.cucumber.adapter.ExtentCucumberAdapter:"**. Create an **extent.properties** file in src/test/resources to configure report path, theme, and other settings. After test execution, the report is generated at the configured path.

Q12. How do you handle environment-specific configurations?

Use a **properties file** (e.g., config.properties) for environments. Create profile-specific files like config.dev.properties, config.qa.properties. Load using System.getProperty("env") or Maven profiles (-Penv=qa). Access values via a ConfigReader utility class in hooks or step definitions.

Q13. What happens if a step definition is missing for a step in feature file?

Cucumber will mark the step as **undefined** and skip it (and subsequent steps). The scenario fails. Cucumber prints a code snippet suggesting the missing step definition. If **strict mode** is enabled, the build fails immediately.

Q14. Can you have multiple Test Runner classes? Why?

Yes. Multiple runners are common to organize execution by: **(1)** environment (DevRunner, QARunner), **(2)** test type (SmokeRunner, RegressionRunner), **(3)** module (LoginRunner, CheckoutRunner), **(4)** parallel execution (each runner runs in a separate thread). This gives better control over test scope.

Q15. How do you skip a scenario conditionally at runtime?

Use **Assumptions** in JUnit (Assume.assumeTrue(condition)) at the start of a step or hook. If false, the scenario is skipped. Alternatively, throw a **org.junit.AssumptionViolatedException** or use Cucumber's @Before hook with a tag check to skip selectively.

Q16. What is the use of the @CucumberOptions monochrome property?

monochrome = true removes color codes (ANSI escape sequences) from console output, making it readable in plain-text logs, CI tools, or text files. Without it, you'll see control characters in non-color-supporting consoles.

Q17. How does Cucumber handle exceptions during step execution?

If an exception is thrown in a step definition: **(1)** The step is marked as **failed**. **(2)** All subsequent steps in the scenario are marked as **skipped**. **(3)** @After hooks still execute (useful for cleanup and screenshots). **(4)** The scenario is reported as failed in the test report.

Q18. What is the difference between Cucumber and Selenium?

Cucumber is a BDD framework for writing readable test cases. **Selenium** is a tool for browser automation. Cucumber doesn't perform any browser interactions on its own - it uses Selenium (or Appium, REST Assured, etc.) inside step definitions to actually drive the application.

Q19. How do you implement a hybrid framework using Cucumber?

A hybrid Cucumber framework combines: **(1)** Cucumber BDD layer (feature files + step definitions). **(2)** Page Object Model for UI maintainability. **(3)** Data-driven testing using Examples / external Excel/JSON. **(4)** Properties file for configuration. **(5)** Reusable utilities (DB, API, file handling). **(6)** Extent/Allure reporting. **(7)** Maven for build and CI/CD integration.

Q20. How can you rerun failed scenarios in Cucumber?

Use the **rerun plugin**: add **"rerun:target/rerun.txt"** to plugins. After execution, failed scenarios are written to rerun.txt. Create a second runner that points **features = "@target/rerun.txt"** to execute only the failed scenarios in a second pass.

PART 2

SoapUI

API Testing Tool for SOAP & REST Services

19. Introduction to Web Services

A **Web Service** is a software system designed to support interoperable machine-to-machine interaction over a network. It allows two applications - possibly written in different languages, running on different platforms - to communicate via standard internet protocols like HTTP.

Key Characteristics

- Platform-independent and language-independent
- Uses standard protocols: HTTP, HTTPS, XML, JSON, SOAP, REST
- Communicates using request/response messaging
- Discoverable through WSDL (SOAP) or OpenAPI/Swagger (REST)
- Stateless (REST) or stateful (SOAP)

Types of Web Services

Type	Protocol	Data Format
SOAP	SOAP over HTTP/HTTPS, SMTP	XML only
REST	HTTP/HTTPS	JSON, XML, plain text, etc.

20. SOAP vs REST

Understanding the difference between SOAP and REST is fundamental for API testing.

Aspect	SOAP	REST
Full Form	Simple Object Access Protocol	Representational State Transfer
Type	Protocol	Architectural Style
Data Format	XML only	JSON, XML, Text, HTML
Transport	HTTP, SMTP, TCP	HTTP/HTTPS only
Standards	WSDL, XSD, WS-Security	OpenAPI/Swagger (optional)
Statefulness	Can be stateful	Stateless
Bandwidth	Heavy (XML envelope)	Lightweight
Caching	Not cacheable	Cacheable
Security	WS-Security (built-in)	HTTPS + tokens (JWT, OAuth)
Best For	Enterprise (banking, payment)	Web/Mobile APIs
Methods	Only POST	GET, POST, PUT, DELETE, PATCH

21. WSDL, XML, XSD, JSON Basics

WSDL (Web Services Description Language)

WSDL is an **XML-based language** that describes a SOAP web service. It defines: **(a)** what operations are available, **(b)** what messages are exchanged, **(c)** what data types are used, and **(d)** where the service is located. It acts like a contract for the service.

Major WSDL elements:

- **<types>** - Data type definitions (usually XSD schemas)
- **<message>** - Defines the data being communicated
- **<portType>** - Set of operations (the service interface)
- **<binding>** - Specifies protocol & data format details
- **<service>** - Endpoint URL of the service

XML (eXtensible Markup Language)

XML is a markup language used to store and transport data. It is the data format used in SOAP messages.

```
<?xml version="1.0"?>
<user>
  <id>101</id>
  <name>John Doe</name>
  <email>john@test.com</email>
</user>
```

XSD (XML Schema Definition)

XSD defines the **structure, content, and data types** of an XML document. It is used to validate XML messages against a defined schema.

JSON (JavaScript Object Notation)

JSON is a lightweight, text-based data format commonly used in REST APIs. It uses key-value pairs and is easy to read and parse.

```
{
  "id": 101,
  "name": "John Doe",
  "email": "john@test.com",
  "roles": ["admin", "user"]
}
```

Sample SOAP Request

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Header/>
  <soap:Body>
    <GetUser>
      <UserId>101</UserId>
    </GetUser>
  </soap:Body>
</soap:Envelope>
```

22. What is SoapUI?

SoapUI is the world's most widely-used open-source API testing tool, developed by SmartBear. It supports testing of **SOAP, REST, GraphQL, and JMS** services. SoapUI provides a graphical

interface for creating, executing, and managing API tests without writing extensive code.

Key Features

- Test SOAP, REST, GraphQL, and JMS services
- Drag-and-drop interface for building test cases
- Built-in assertions for validating responses
- Property transfer between test steps
- Groovy scripting for custom logic
- Data-driven testing using Excel/CSV (Pro)
- Mock services for simulating APIs
- Security testing (SQL injection, XSS, etc.)
- Load and performance testing (LoadUI integration)
- Comprehensive reporting

SoapUI Editions

Feature	SoapUI Open Source	ReadyAPI (Pro)
SOAP/REST Testing	Yes	Yes
Basic Assertions	Yes	Yes
Data-Driven Testing	Limited (Groovy)	Built-in DataSource
Mock Services	Yes	Advanced
Security Testing	Limited	Comprehensive
Load Testing	Basic	Advanced (LoadUI)
CI/CD Integration	Limited	Full
Cost	Free	Paid (license)

23. Installation & First Project

To install SoapUI Open Source:

- Visit **www.soapui.org** and download the latest version
- Choose installer for your OS (Windows, Mac, Linux)
- Run the installer and follow on-screen instructions
- Launch SoapUI - you'll see the workspace window
- Workspaces contain projects, which contain test suites and services

SoapUI Workspace Structure

```
Workspace
  ■■■ Project
       ■■■ Service / Interface (SOAP/REST endpoint)
```

```
■    ■■■ Operations / Resources
■         ■■■ Sample Requests
■■■ Test Suites
■    ■■■ Test Cases
■         ■■■ Test Steps
■              ■■■ SOAP/REST Request
■              ■■■ Assertions
■              ■■■ Property Transfer
■              ■■■ Groovy Script
■■■ Mock Services
```

24. SOAP Project from WSDL

Creating a SOAP project in SoapUI:

- **File > New SOAP Project**
- Enter **Project Name**
- Paste the **WSDL URL** (e.g., http://www.dneonline.com/calculator.asmx?WSDL)
- Check 'Create sample requests for all operations'
- Click **OK**

SoapUI will parse the WSDL and create the service structure with all operations. You can expand each operation to see auto-generated sample SOAP requests. Modify the request body, then click the green **Submit** arrow to send and view the response.

25. REST Project Setup

Creating a REST project in SoapUI:

- **File > New REST Project**
- Enter the **endpoint URL** (e.g., https://reqres.in/api/users)
- Click **OK** - SoapUI creates the project with a Request
- Choose HTTP method: **GET, POST, PUT, DELETE, PATCH**
- Add request body (JSON/XML) for POST/PUT
- Add query parameters / headers as needed
- Click **Submit** to send the request

HTTP Methods Cheat Sheet

Method	Purpose	Idempotent?
GET	Retrieve a resource	Yes
POST	Create a new resource	No
PUT	Update/replace a resource	Yes
PATCH	Partially update a resource	No (usually)

Method	Purpose	Idempotent?
DELETE	Delete a resource	Yes

26. Test Suites, Test Cases, Test Steps

SoapUI organizes tests in a hierarchical structure:

Level	Description
Test Suite	Top-level container for related Test Cases
Test Case	A single test scenario containing multiple Test Steps
Test Step	Individual action (send request, assertion, script, transfer)

Types of Test Steps

- **SOAP Request / REST Request** - Sends an API call
- **Property Transfer** - Transfers data between test steps
- **Groovy Script** - Runs custom code for complex logic
- **Conditional GoTo** - Branches execution based on conditions
- **Delay** - Pauses execution for a specified time
- **JDBC Request** - Executes a SQL query against a database
- **Run Test Case** - Executes another test case (reuse)
- **Properties** - Defines variables used in the test case
- **Manual Test Step** - For documentation purposes

27. Assertions - All Types

Assertions validate that the API response matches expected criteria. Without assertions, a test only verifies that a response was received, not that it's correct.

Common Assertion Types

Assertion	Purpose
Contains	Checks if response contains a specific text/regex
Not Contains	Verifies that a text is NOT present in response
XPath Match	Validates a specific XPath value in XML response
XQuery Match	Validates XQuery expression result
JSONPath Match	Validates a JSONPath value in JSON response
JSONPath Count	Counts elements matching a JSONPath
Valid HTTP Status Codes	Checks response code (e.g., 200, 201)

Assertion	Purpose
Invalid HTTP Status Codes	Verifies response code is NOT in a list
Response SLA	Checks if response time is within a limit
Schema Compliance	Validates response against XSD/JSON Schema
SOAP Fault	Checks if response contains a SOAP Fault
Not SOAP Fault	Verifies absence of a SOAP Fault
Script Assertion	Custom Groovy code for complex validation

Adding an Assertion

- Open the response of a test step
- Click the **Assertions** tab at the bottom
- Click the **+ (Add Assertion)** button
- Select the assertion type and configure it
- Save and re-run to validate

Example: JSONPath Match

```
Response:
{
  "status": "success",
  "data": {
    "id": 101,
    "name": "John"
  }
}

JSONPath Expression: $.data.name
Expected Value: John
```

28. Property Transfer

Property Transfer is used to extract a value from one test step's response and pass it to another test step's request. This is essential for chained API calls (e.g., create user, then use the returned user ID in a subsequent call).

Example Workflow

- **Step 1 (POST /users):** Create a user - response contains the userId
- **Step 2 (Property Transfer):** Extract **$.id** from Step 1's response
- **Step 3 (GET /users/{id}):** Use the transferred userId in the URL

Configuration

In the Property Transfer step:

- **Source:** Select the previous request and choose **Response** + JSONPath / XPath
- **Target:** Select the next request and choose property/parameter to set

- Use expressions like **$.id** (JSONPath) or **//user/id** (XPath)

29. Groovy Scripting

Groovy is a JVM-based scripting language used in SoapUI for: **(a)** custom assertions, **(b)** dynamic data generation, **(c)** setup/teardown scripts, **(d)** property manipulation, **(e)** file I/O and database operations.

Common Groovy Use Cases

Log a message

```
log.info "Hello from Groovy"
```

Get a test step property

```
def projectName = context.expand('${#Project#}')
def userId = testRunner.testCase.getTestStepByName("CreateUser")
              .getPropertyValue("Response")
log.info "User ID: " + userId
```

Set a property dynamically

```
def randomNum = (int)(Math.random() * 10000)
testRunner.testCase.setPropertyValue("uniqueId", randomNum.toString())
```

Read response and validate

```
import groovy.json.JsonSlurper

def response = context.expand('${GetUser#Response}')
def json = new JsonSlurper().parseText(response)

assert json.status == "success"
assert json.data.id != null
log.info "User name: " + json.data.name
```

Read data from CSV file

```
def file = new File("C:/data/users.csv")
file.eachLine { line ->
    def cols = line.split(",")
    log.info "User: " + cols[0]
}
```

30. Data-Driven Testing

Data-Driven Testing (DDT) means running the same test with multiple input data sets to validate behavior across various scenarios.

Approaches in SoapUI

- **Open Source:** Use Groovy script to read from CSV/Excel and loop through data
- **ReadyAPI (Pro):** Built-in **DataSource** and **DataSource Loop** steps
- **Properties File:** Store test data and reference via ${#Project#propertyName}

DDT with Groovy + CSV

```
// Read CSV and run test for each row
def file = new File("C:/data/testdata.csv")
def lines = file.readLines()

// Skip header
for (int i = 1; i < lines.size(); i++) {
    def cols = lines[i].split(",")
    def username = cols[0]
    def password = cols[1]

    // Set as test case properties
    testRunner.testCase.setPropertyValue("username", username)
    testRunner.testCase.setPropertyValue("password", password)

    // Run the login test step
    testRunner.runTestStepByName("LoginRequest")
}
```

31. Mock Services

A **Mock Service** in SoapUI simulates the behavior of a real web service. It returns predefined responses to incoming requests. Mocking is critical when:

- The actual service is not yet developed (parallel development)
- The service is unstable or unavailable
- You need to test edge cases (errors, timeouts) hard to reproduce
- Third-party services charge per call (cost optimization)
- Testing in environments where external services aren't accessible

Creating a Mock Service

- Right-click on the Service / Interface > **Generate SOAP Mock Service**
- Configure the port and path
- Add Mock Responses for each operation
- Customize response content and headers
- Click the **Start** button to run the mock
- Point your client/test to the mock URL

Dynamic Mock Responses

Use Groovy in the **Dispatch** tab to return different responses based on the incoming request (e.g., based on a request parameter or header).

32. Security Testing

SoapUI supports security scans to identify common API vulnerabilities. Open the test case and add a **Security Test**.

Common Security Tests

Test	Purpose
SQL Injection	Tries to inject SQL commands in input parameters
XPath Injection	Tries XPath injection on XML inputs
XML Bomb	Sends huge nested XML to test parser stability
Cross-Site Scripting (XSS)	Injects scripts to detect script execution
Boundary Scan	Tests min/max/empty values in parameters
Invalid Types	Sends mismatched data types (string for int)
Malformed XML	Sends broken XML to test error handling
Sensitive Data Exposure	Looks for sensitive info in responses

33. Load Testing

SoapUI lets you convert any functional test case into a **Load Test** to evaluate performance under different loads.

Creating a Load Test

- Right-click on a Test Case > **New Load Test**
- Configure number of **threads** (concurrent users)
- Set **test duration** or limit
- Choose a **load strategy**
- Run and analyze statistics

Load Strategies

Strategy	Behavior
Simple	Constant load with fixed thread count
Fixed Rate	Sends requests at a fixed rate (TPS)
Variance	Randomly varies the thread count over time
Burst	Periodic bursts of high load with idle periods
Thread	Gradually increases threads to find max capacity
Ramp Up	Gradually adds users until the target is reached

Key Performance Metrics

- **min/max/avg response time** - Latency statistics
- **tps** - Transactions per second (throughput)
- **bytes** - Data transferred
- **cnt** - Number of requests processed

- **err** - Number of errors
- **rat** - Average response time over the last X seconds

34. L1 Interview Questions (SoapUI)

Fundamental questions about SoapUI and API testing.

Q1. What is SoapUI?

SoapUI is an open-source API testing tool developed by SmartBear, used to test SOAP and REST web services. It provides a GUI for creating, running, and managing functional, regression, load, and security tests.

Q2. What is the difference between SOAP and REST?

SOAP is a protocol that uses XML for messages and is stricter (WSDL contract, WS-Security). **REST** is an architectural style using HTTP methods (GET, POST, PUT, DELETE) and supports multiple formats like JSON and XML. REST is lighter, faster, and used widely for web/mobile APIs.

Q3. What is WSDL?

WSDL (Web Services Description Language) is an XML-based file that describes a SOAP web service - what operations are available, what data types it uses, and where the service is located. It serves as a contract between the service and consumers.

Q4. What is an API?

An API (Application Programming Interface) is a set of rules that allows software applications to communicate with each other. It defines how requests are made, what data is exchanged, and what responses are returned.

Q5. What are the main HTTP methods used in REST?

GET (read), **POST** (create), **PUT** (update/replace), **PATCH** (partial update), **DELETE** (remove), **HEAD** (metadata only), **OPTIONS** (get supported methods).

Q6. What are common HTTP status codes?

2xx (Success): 200 OK, 201 Created, 204 No Content. **3xx (Redirect):** 301 Moved, 304 Not Modified. **4xx (Client Error):** 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 405 Method Not Allowed. **5xx (Server Error):** 500 Internal Server Error, 502 Bad Gateway, 503 Service Unavailable.

Q7. What is the difference between XML and JSON?

XML uses tags, is verbose, and supports namespaces and attributes. **JSON** uses key-value pairs, is lightweight, easier to read, and natively parsed by JavaScript. JSON is more common in REST APIs; XML is required by SOAP.

Q8. What is a Test Suite in SoapUI?

A Test Suite is a top-level container in SoapUI that groups related test cases. It allows organizing tests by feature, module, or environment.

Q9. What is a Test Case in SoapUI?

A Test Case is a single test scenario within a Test Suite. It contains multiple Test Steps that execute sequentially to verify a specific functionality.

Q10. What is a Test Step in SoapUI?

A Test Step is an individual action in a test case - sending a request, transferring properties, executing a Groovy script, running a database query, etc.

Q11. What types of assertions are available in SoapUI?

Common assertions: Contains, Not Contains, XPath Match, XQuery Match, JSONPath Match, JSONPath Count, Valid HTTP Status Codes, Response SLA, Schema Compliance, SOAP Fault, and Script Assertion.

Q12. What is an endpoint in SoapUI?

An **endpoint** is the URL where the web service is hosted (e.g., http://api.example.com/users). SoapUI sends requests to this endpoint.

Q13. How do you create a SOAP project in SoapUI?

File > New SOAP Project > Enter project name > Paste the WSDL URL > Click OK. SoapUI auto-generates the service structure and sample requests for all operations.

Q14. How do you create a REST project in SoapUI?

File > New REST Project > Enter the API endpoint URL > Click OK. Set the HTTP method, add headers/params/body, then Submit.

Q15. What is a Property in SoapUI?

A **Property** is a variable that can be defined at multiple levels (project, test suite, test case, test step) and used in requests, assertions, or scripts using ${#Level#propertyName} syntax.

Q16. How do you pass data between test steps?

Use the **Property Transfer** test step. It extracts a value from one step's response (using XPath / JSONPath) and assigns it to another step's request property.

Q17. What is Groovy in SoapUI?

Groovy is a JVM scripting language used in SoapUI for custom logic - dynamic data, custom assertions, file/database operations, and conditional flow.

Q18. What is a Mock Service?

A Mock Service simulates a real web service by returning predefined responses. It's useful when the actual service isn't available or for testing edge cases.

Q19. What is SOAP envelope?

The SOAP envelope is the root XML element of a SOAP message. It contains: **Header** (optional metadata, security tokens) and **Body** (the actual request or response payload).

Q20. What is the use of an Assertion in SoapUI?

Assertions validate that the response matches the expected behavior. Without assertions, the test only verifies that a response was received, not that it's correct.

35. L2 Interview Questions (SoapUI)

Intermediate questions on real testing scenarios and integrations.

Q1. How do you handle dynamic data in SoapUI?

Multiple ways: **(1)** Use SoapUI's built-in functions like ${=java.util.UUID.randomUUID()} or ${=System.currentTimeMillis()}. **(2)** Use Groovy scripts to generate dynamic values and store in properties. **(3)** Use Property Transfer to pass values from prior responses. **(4)** Use a property file or data source for parametrized values.

Q2. How do you do data-driven testing in SoapUI Open Source (free)?

Use a **Groovy Script** step to read data from CSV/Excel/database and loop through rows. For each iteration, set the test case properties and call testRunner.runTestStepByName() to execute the target steps. In **ReadyAPI (Pro)**, the DataSource and DataSource Loop steps simplify this without scripting.

Q3. How do you handle authentication in SoapUI?

SoapUI supports: **(1) Basic Auth** - Auth tab > Authorization Type Basic. **(2) OAuth 2.0** - Auth tab > OAuth 2.0 (configure flow, token, scopes). **(3) API Key** - Pass as a header or query parameter. **(4) WS-Security** (SOAP) - Project > WS-Security Configurations. **(5) Bearer Token** - Add 'Authorization: Bearer ${token}' in Headers.

Q4. How do you validate JSON response in SoapUI?

Use **JSONPath Match** assertion. Enter JSONPath expression (e.g., $.data.id) and the expected value. For counting elements, use **JSONPath Count**. For complex validations, use a **Script Assertion** with JsonSlurper to parse and assert programmatically.

Q5. How do you validate XML response?

Use **XPath Match** for specific node values (e.g., //user/id). Use **XQuery Match** for more complex XML queries. Use **Schema Compliance** to validate against the XSD.

Q6. How do you chain API calls in SoapUI?

Use **Property Transfer** between test steps. Step 1 sends a request and gets a response. Property Transfer extracts a value from Step 1's response (e.g., an ID). Step 2 uses this value in its request. This pattern is essential for end-to-end workflows.

Q7. How do you read data from a file using Groovy in SoapUI?

Use Groovy's File class:

```
def file = new File("C:/data.csv")
file.eachLine { line -> log.info line }
```

For Excel, use Apache POI (add jar to SoapUI's bin/ext folder).

Q8. How do you run a Groovy assertion?

Add a **Script Assertion** to a request. Write Groovy code that throws an AssertionError if conditions aren't met. Example: `assert messageExchange.responseContent.contains("success")`. Use 'context', 'log', and 'messageExchange' built-in variables.

Q9. How do you integrate SoapUI with Jenkins or CI/CD?

Use the **testrunner.bat** (Windows) or **testrunner.sh** (Linux) command-line tool provided with SoapUI. Example command: `testrunner.bat -s"TestSuite" -c"TestCase" "project.xml"`. In Jenkins, create a build step that executes this. Alternatively, use the Maven

SoapUI plugin to run tests as part of a Maven build.

Q10. How do you generate reports in SoapUI?

In SoapUI Open Source: use command-line testrunner with options like -f (output folder) and -j (JUnit-style XML report) or -r (printable HTML). Custom reports can be built using Groovy to write to files. In ReadyAPI (Pro), rich built-in HTML reports are available.

Q11. What is the difference between Property Transfer and Groovy Script?

Property Transfer is a declarative step for simple data movement (extract value & assign to property). **Groovy Script** is a code-based step for complex logic - conditionals, loops, calculations, file I/O, multiple property updates. Use Property Transfer for simple flows and Groovy for advanced scenarios.

Q12. How do you handle SSL certificates in SoapUI?

Go to **File > Preferences > SSL Settings**. Provide the keystore (.jks or .p12) path, password, and choose a default certificate. SoapUI then uses these credentials when calling HTTPS endpoints requiring client-side SSL authentication.

Q13. What is the difference between SoapUI Open Source and ReadyAPI?

SoapUI Open Source is free with basic SOAP/REST testing and Groovy-based extensibility. **ReadyAPI** is the paid commercial version with advanced features: DataSource/DataSink for data-driven testing, comprehensive security testing, advanced load testing (LoadUI), full CI/CD plugins, sophisticated mocking, code-free assertions, and enterprise support.

Q14. How can you do parameterization in SoapUI?

Approaches: **(1)** Define **properties** at project/suite/case level and reference with ${#Project#prop} syntax. **(2)** Use the **Properties** test step to define case-level properties. **(3)** Use a **property file** loaded at runtime. **(4)** Use **data sources** (Pro) or Groovy + CSV in Open Source.

Q15. How do you simulate different responses in a mock service?

In the Mock Service, the **Dispatch** options support multiple strategies: **SEQUENCE** (returns mock responses in order), **RANDOM** (random response), **QUERY_MATCH** (based on XPath/JSONPath in incoming request), **SCRIPT** (Groovy logic to choose response). The SCRIPT option is the most flexible for conditional mocking.

Q16. How do you handle large XML/JSON responses?

Use specific XPath/JSONPath assertions instead of full Contains assertions. For very large responses, increase JVM memory in SoapUI's vmoptions file. Use Groovy to parse and process only the needed portions. Consider streaming parsers (StAX for XML) for huge payloads.

Q17. What is the use of testrunner.bat?

testrunner.bat (Windows) / .sh (Linux/Mac) is the command-line execution tool for SoapUI. It runs projects/suites/test cases outside the SoapUI GUI - essential for CI/CD integration and headless execution. Common options: -s (suite), -c (case), -r (report), -f (output folder), -P (set property).

Q18. How do you debug a failing test in SoapUI?

Steps: **(1)** Open the request and view the full response in the response panel. **(2)** Check the **Raw** tab for the actual HTTP traffic. **(3)** Review log panels (SoapUI log, Script log, Error log). **(4)** Add **log.info** statements in Groovy. **(5)** Run test step-by-step instead of full case. **(6)** Verify property values via 'Properties' panel after each step.

Q19. How do you connect a database in SoapUI?

Add a **JDBC Request** test step. Configure: **(a)** Driver class (e.g., `com.mysql.cj.jdbc.Driver`), **(b)** Connection URL, **(c)** Username/Password, **(d)** SQL query. Add JDBC assertions to validate results. Place the JDBC driver JAR in SoapUI's bin/ext folder.

Q20. How do you handle environments (Dev, QA, Prod) in SoapUI?

Use **Environments** (right-click Project > Environments) to define multiple sets of endpoints, properties, and database connections. Switch between Dev/QA/Prod from the environment dropdown. Alternatively, parameterize endpoints with properties and load from environment-specific property files (Open Source approach).

Final Tips for Your Interview

Before the interview:
• Practice writing a sample feature file and step definitions from scratch
• Build a simple Cucumber + Selenium project locally and run it
• Create a SoapUI project, add assertions, and use Property Transfer
• Be comfortable explaining the project structure on a whiteboard

During the interview:
• Use clear examples - mention specific scenarios from your project
• If unsure, talk through your thinking - shows problem-solving
• Don't memorize - understand the *why* behind each concept
• Connect concepts: Cucumber + Selenium, SoapUI + JDBC, etc.

Common follow-up areas:
• Selenium WebDriver basics (locators, waits, frames)
• Java basics (collections, OOP, exception handling)
• Maven / Git / Jenkins
• Manual testing fundamentals (test cases, defect lifecycle)
• SQL basics (joins, queries, validations)

All the best for your interview!